HIGH

Honest Validator Self-Equivocation: SkipVote + NotarizeVote for the Same Slot

Repo: ton-blockchain/ton **Branch:** testnet **Commit:** `3bb6abc` **Scope:** Primary — validator/consensus/simplex/ **Date:** 2026-03-14

Summary

Three linked bugs in `consensus.cpp` and `pool.cpp` allow an honest validator to broadcast both a `SkipVote` and a `NotarizeVote` for the same slot during normal operation. The equivocation is silently ignored by all peers — no `MisbehaviorReport` is generated. Under conditions where block validation exceeds the alarm interval, every validator equivocates simultaneously, causing complete liveness failure for the affected consensus rounds.

Root Cause

Bug A — alarm() uses insufficient guard

`ConsensusImpl::alarm()` broadcasts `SkipVote` for any slot where `voted_final = false`, without checking whether the validator has already committed to notarizing that slot:

```
// consensus.cpp — alarm(), lines 144-150
for (td::uint32 i = range_start; i < window_end; ++i) {
  auto slot = state_->slot_at(i);
  if (slot && !slot->state->voted_final) {  // BUG: must also check !voted_notar &&
!voted_skip
    owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
    slot->state->voted_skip = true;
  }
}
```

Bug B — try_notarize() does not check voted_skip after resuming

`try_notarize()` is a coroutine with three `co_await` suspension points. The `voted_notar` flag is set only after all three complete. If `alarm()` fires during the `ValidationRequest` suspension, `voted_skip = true` is set, but the coroutine resumes without checking it:

```
// consensus.cpp — try_notarize() (after co_await ValidationRequest resumes)
auto validation_result = co_await owning_bus().publish<ValidationRequest>(...);
// alarm() can fire here: voted_skip = true, SkipVote broadcast
// BUG B: no check for voted_skip before the next two lines
slot.state->voted_notar = candidate->id;
owning_bus().publish<BroadcastVote>(NotarizeVote{candidate->id}).start().detach();
```

Race sequence:

1. Candidate arrives → `try_notarize(slot N)` starts, suspends at `co_await ValidationRequest`

2. Alarm fires → slot N has `voted_final=false` → broadcasts `SkipVote{N}`, sets `voted_skip=true`
3. Validation completes → coroutine resumes → broadcasts `NotarizeVote{N}`

Both votes are now in flight from the same validator.

Bug C — check_invariants() does not detect NotarizeVote + SkipVote combination

`Tsentrizbirkom::check_invariants()` in `pool.cpp` detects two conflicting vote combinations but is missing the third:

```
// pool.cpp — check_invariants()
if (notarize_.has_value() && finalize_.has_value() && notarize_->vote.id != finalize_-
>vote.id)
  return ConflictingVotes::create(...);    // ✓ detected
if (finalize_.has_value() && skip_.has_value())
  return ConflictingVotes::create(...);    // ✓ detected
// MISSING: NotarizeVote + SkipVote is also a conflicting combination in Simplex
// if (notarize_.has_value() && skip_.has_value()) → ConflictingVotes   ← not present
```

The equivocating validator's votes are accepted without any `MisbehaviorReport`, and its weight is counted towards both `notarize_weight` and `skip_weight` for the same slot.

Demonstrated Impact

Tested on commit `3bb6abc` using the in-tree `test-consensus` binary with 8 equal-weight validators, 1.5–2.5 s validation time, 200 ms block target (requires Clang 18+ to build — GCC cannot compile this codebase):

PoC output — pool.cpp:525 WARNING:

`Dropping NotarizeVote{id={60, +7z0gpDDmAsyzTc3oLsh7f+82mx3OuXOMCuNHuCwvOM=}} from validator 2 at lzPRai9kr3RHHmjlDhvucvghdTtfT0W1EsHWV7G+Muk= which references a finalized slot`

Slot 60 was already finalised via `SkipCert` when this `NotarizeVote` arrived, proving validator 2 broadcast both vote types for the same slot. Identical pattern observed across all 8 validators.

523 confirmed equivocating (validator, slot) pairs observed in a single 30-second run. Equivocating validators have `voted_skip=true`, which blocks `try_vote_final()` from issuing `FinalizeVote` — every candidate block for an affected slot is abandoned. The result is severe throughput degradation across the run.

Reproduction

```
git clone --branch testnet https://github.com/ton-blockchain/ton.git
cd ton && mkdir build-clang && cd build-clang
CC=clang CXX=clang++ cmake .. -DCMAKE_BUILD_TYPE=Release -DTON_USE_ROCKSDB=OFF
cmake --build . --target test-consensus -j$(nproc)

# Clang 18+ required
./test/consensus/test-consensus \
  --validation-time 1.5:2.5 \
```

```
  --target-rate-ms 200 \
  --n-nodes 8 \
  --duration 30 \
  -v 2 2>&1 | grep "Dropping NotarizeVote.*finalized slot"
```

Expected output: multiple `WARNING [pool.cpp:525]` lines showing `NotarizeVote` arrivals for already-skipped slots, confirming each validator sent both vote types.

Fix

Bug A — add guards in `alarm()`:

```
if (slot && !slot->state->voted_final
         && !slot->state->voted_notar    // new
         && !slot->state->voted_skip) {  // new
  owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
  slot->state->voted_skip = true;
}
```

Bug B — check `voted_skip` in `try_notarize()` after all suspension points:

```
auto validation_result = co_await owning_bus().publish<ValidationRequest>(...);
if (slot.state->voted_skip) { co_return {}; }  // new guard
slot.state->voted_notar = candidate->id;
owning_bus().publish<BroadcastVote>(NotarizeVote{candidate->id}).start().detach();
```

Bug C — add the missing case to `check_invariants()` in `pool.cpp`:

```
if (notarize_.has_value() && skip_.has_value()) {
  return ConflictingVotes::create(
    notarize_->serialize_as_proof(), skip_->serialize_as_proof());
}
```

TON Blockchain Contest Round 2 — Primary scope finding. Submitted by TesseraeVentures.